

Log In ▾

The wiki page is under active construction, expect bugs.

Základní algoritmy a datové struktury pro vyhledávání a řazení. Vyhledávací stromy, rozptylovací tabulky. Prohledávání grafu. Úlohy dynamického programování. Asymptotická složitost a její určování.

vector2 ▾

B4B33ALG Webové stránky předmětu

- **Řád růstu funkcí** – asymptotická složitost algoritmu.
- **Rekurze** – stromy, binární stromy, prohledávání s návratem.
- **Fronta, zásobník** – průchod stromem/grafem do šířky/hloubky.
- **Binární vyhledávací stromy** – AVL a B-stromy.
- **Algoritmy řazení** – Insert Sort, Selection Sort, Bubble Sort, QuickSort, Merge Sort, Halda, Heap Sort, Radix Sort, Counting Sort.
- **Dynamické programování** – struktura optimálního řešení, odstranění rekurze. Nejdelší společná podposloupnost, optimální násobení matic, problém batohu.
- **Rozptylovací tabulky (Hashing)** – hashovací funkce, řešení kolizí, otevřené a zřetězené tabulky, double hashing, srůstající tabulky, univerzální hashování.

1. Řád růstu funkcí

- asymptotická složitost algoritmu
- slouží k odhadu doby běhu algoritmu vzhledem k velikosti vstupu

Asymptotická složitost

Je způsob klasifikace algoritmů podle chování při rostoucím množství vstupních dat. Důležité pro srovnání algoritmů nezávisle na implementaci a výpočetní síle.

Horní asymptotický odhad (velké O): $f(n) \in O(g(n)) \rightarrow$ funkce f je shora ohraničena funkcí g , až na konstantu. Formálně: $\exists c > 0, n_0 > 0 : \forall n > n_0 : f(n) \leq c \cdot g(n)$

Dolní asymptotický odhad (velká Ω): $f(n) \in \Omega(g(n)) \rightarrow$ funkce f je zdola ohraničena funkcí g . Formálně: $\exists c > 0, n_0 > 0 : \forall n > n_0 : f(n) \geq c \cdot g(n)$

Těsný asymptotický odhad (velké Θ): $f(n) \in \Theta(g(n)) \rightarrow$ funkce f je ohraničena funkcí g shora i zdola. Formálně: $f(n) \in O(g(n))$ a zároveň $f(n) \in \Omega(g(n))$

Poznámka: Třída složitosti obsahuje všechny algoritmy s "podobným" chováním až na konstantu.

$\rightarrow$ Jinými slovy: asymptotická složitost nám říká, jak se bude algoritmus chovat pro opravdu velké vstupy. Nezáleží na detailech, implementaci nebo výkonu stroje – zajímá nás jen „trend“ růstu počtu operací.

Klasifikace dle složitosti

Od nejmenší po největší:

- $O(1)$ – konstantní
- $O(\log n)$ – logaritmická
- $O(n)$ – lineární
- $O(n \log n)$ – lineárně logaritmická

- $O(n^2)$ – kvadratická
- $O(n^3)$ – kubická
- $O(n^k)$ – polynomiální
- $O(k^n)$ – exponenciální
- $O(n!)$ – faktoriálová

→ Čím „nižší“ složitost, tím lépe škáluje. Např. algoritmus s $O(n)$ zpracuje dvojnásobný vstup asi dvakrát pomaleji, ale $O(n^2)$ už čtyřikrát.

2. Rekurze

- Stromy, binární stromy, prohledávání s návratem

Stromy

Strom je základní datová struktura, která se skládá z kořene, uzlů a listů. Používá se pro uchovávání hierarchických dat, jako jsou například složky v počítači nebo organizační struktury.

- Kořen je výchozí uzel stromu – ten, který nemá žádného rodiče.
- Uzel je obecně jakýkoliv prvek stromu. Může mít žádného, jednoho nebo více potomků.
- List je uzel, který nemá žádné potomky. Často představuje koncový bod datové struktury.

Strom je speciálním případem grafu – je to souvislý, neorientovaný graf bez cyklů, ve kterém mezi každými dvěma uzly existuje právě jedna cesta.

- Vždy platí, že strom s n uzly má přesně $n - 1$ hran.
- Hloubka stromu je délka nejdelší cesty od kořene k některému listu. Ukazuje, jak „vysoký“ strom je.

V informatice se často používá tzv. kořenový strom, což je strom, kde začínáme právě od jednoho výchozího kořenového uzlu.

- Stromy tedy nejsou jen teoretická struktura – často se s nimi setkáváme i při běžné práci s daty nebo při návrhu algoritmů.

Binární stromy

Binární strom je strom, kde každý uzel má maximálně dva potomky. Tito potomci se obvykle nazývají levý a pravý. Tato jednoduchá struktura je ale velmi výkonná a často používaná.

- Binární vyhledávací strom (BST) je binární strom s uspořádanými hodnotami – pro každý uzel platí:
 - hodnota v levém podstromu je menší než hodnota v uzlu,
 - hodnota v pravém podstromu je větší než hodnota v uzlu.
- Vyvážené BST (např. AVL stromy nebo červeno-černé stromy) udržují výšku stromu malou, obvykle logaritmickou, což zajišťuje rychlé operace jako vyhledávání, vkládání a mazání.

Reprezentace v jazyce C:

```
typedef struct node {
    int data;
    struct node *left;
    struct node *right;
} Node;
```

- Ve většině případů se binární stromy implementují pomocí dynamických struktur – tedy uzly spojované pomocí ukazatelů. Alternativně lze binární strom uložit i jako pole (například v algoritmu heapsort).

Využití

Stromy se široce využívají k reprezentaci hierarchických informací:

- souborové systémy (adresáře a soubory),
- organizační struktury firem,

- výrazy v programovacích jazycích (každý uzel je operace, potomci jsou operandy),
- datové struktury jako haldy, binární vyhledávací stromy a rozhodovací stromy.

Také se využívají k realizaci algoritmů, např. při prohledávání, řazení nebo optimalizaci.

Procházení stromem

Jednou z nejčastějších operací nad stromy je jejich průchod – tedy návštěva všech uzlů. Nejčastěji se k tomu používá rekurze.

Existují tři klasické způsoby průchodu binárního stromu:

- Preorder (předřazení) – nejprve navštívíme samotný uzel, poté levý podstrom a nakonec pravý podstrom.
- Inorder (meziřazení) – nejprve levý podstrom, pak uzel a nakonec pravý podstrom.
- Postorder (pořadí) – nejprve levý podstrom, potom pravý podstrom a až nakonec samotný uzel.
- Především inorder je velmi důležitý pro binární vyhledávací stromy – tento způsob průchodu totiž dává prvky seřazené podle velikosti.

Rekurze

Rekurze je programovací technika, kdy funkce volá sama sebe, aby vyřešila menší instanci téhož problému. Je velmi vhodná pro strukturované problémy – jako jsou stromy nebo dělení na menší části.

Důležitou podmínkou každé rekurze je tzv. **ukončovací podmínka**, která zabrání nekonečnému volání – tedy přetečení zásobníku.

Rekurze se typicky používá v technice **rozděl a panuj**:

- Problém se rozdělí na několik menších podobných částí.
- Tyto části se řeší samostatně, opět rekurzivně.
- Výsledky se nakonec spojí do finálního řešení.

Příklady využití rekurze:

- třídící algoritmy: MergeSort, QuickSort,
- výpočty: faktoriál, Fibonacciho čísla,
- algoritmy nad stromy a grafy.

Backtracking (prohledávání s návratem)

Backtracking je technika, která prochází všechny možné kombinace, ale inteligentně vynechává ty, které už zjevně nevedou ke správnému řešení. Zkoušíme řešení krok za krokem a když zjistíme, že už nemůže být platné, vracíme se zpět a zkusíme jinou větev.

Tato metoda je známá z problémů jako jsou n -dámy nebo sudoku. Umí ušetřit obrovské množství času oproti čistému brute-force.

Aby backtracking fungoval, musí být splněno:

- Můžeme zkontrolovat, zda částečné řešení je zatím platné.
 - Např. při n -queens lze i s neúplnou šachovnicí ověřit, jestli se královny neohrožují.
 - Naproti tomu při hledání hashe hesla nemůžeme říct, že je „částečně správně“.
- Problém má **monotónní** strukturu – když už je část řešení špatně, další volby to nespraví.

Jak takový algoritmus vypadá?

```
def candidates(solution):
    # z předchozího řešení vygenerujeme další možné volby
    ...
    return candidates

def add(candidate, solution):
    # Přidání kandidáta do řešení
```

```

...
return new_solution

def remove(candidate, new_solution):
    # Odebrání kandidáta, krok zpět
    ...
    return old_solution

def is_valid(solution):
    # Ověříme platnost řešení
    ...
    return True/False

def process(solution):
    # Hotové řešení nějak zpracujeme
    print(solution)

def backtracking(solution):
    if solution is complete:
        process(solution)
    else:
        for next in candidates(solution):
            if is_valid(next):
                solution = add(next, solution)
                backtracking(solution)
                solution = remove(next, solution)

```

- Použití: n-queens, sudoku, kombinatorické generování, obchodní cestující, a mnoho dalších úloh.

Mistrovská věta

Když máme rekurzivní algoritmus, chceme často zjistit jeho časovou složitost. Pokud má tvar:

$T(n) = a \cdot T(n/b) + f(n)$ můžeme použít tzv. **mistrovskou větu**, která nám umožní složitost určit rychle – bez nutnosti řešit rekurentní rovnici ručně.

- a – kolik podproblémů vznikne
- b – jak moc se velikost zmenší
- $f(n)$ – jak náročné je zpracování mimo rekurzi

Možnosti:

- Pokud $f(n)$ roste pomaleji než $n^{\log_b a} \Rightarrow$ první případ $\Rightarrow T(n) \in \Theta(n^{\log_b a})$
- Pokud $f(n) \sim n^{\log_b a} \Rightarrow$ druhý případ $\Rightarrow T(n) \in \Theta(n^{\log_b a} \cdot \log n)$
- Pokud $f(n)$ roste rychleji a splní se ještě další podmínka $\Rightarrow$ třetí případ $\Rightarrow T(n) \in \Theta(f(n))$

Příklad:

- MergeSort: $T(n) = 2T(n/2) + n$
- $a = 2, b = 2, \log_b a = 1$
- $f(n) = n \in \Theta(n) \Rightarrow$ druhý případ
- $\Rightarrow$ Složitost: $T(n) \in \Theta(n \log n)$

Substituční metoda

Další způsob, jak určit časovou složitost, je tzv. substituční metoda. Používá se k **důkazu** pomocí matematické indukce.

Jak postupovat:

- Odhadneme složitost $T(n) \in O(g(n))$
- A pak to dokážeme indukcí

Příklad:

Odhad: $T(n) = 2T(n/2) + n \in O(n \log n)$

Pak dokážeme, že:

$$T(n) \leq 2 \cdot c \cdot \frac{n}{2} \log \frac{n}{2} + n = c \cdot n \log n - c \cdot n + n$$

Stačí zvolit vhodnou konstantu c a dostaneme požadovanou nerovnost.

- Tato metoda je užitečná, když máme podezření na složitost, ale chceme ji formálně potvrdit.

Rekurzivní strom

Tato metoda spočívá v tom, že si celý průběh rekurze nakreslíme jako strom a sečteme náklady na všech úrovních.

Příklad: $T(n) = 3T(n/4) + n$

- První úroveň: n
- Druhá úroveň: $3 \cdot (n/4) = 3n/4$
- Třetí úroveň: $9 \cdot (n/16) = 9n/16$

To je geometrická řada:

$$T(n) = n \cdot \left(1 + \frac{3}{4} + \left(\frac{3}{4} \right)^2 + \dots \right) = n \cdot \frac{1}{1 - 3/4} = 4n$$

⇒ Celková složitost $T(n) \in O(n)$

- Tato metoda dává pěknou vizuální představu o tom, kde se v rekurzivním algoritmu dělá nejvíc práce.

Shrnutí

- Stromy jsou skvělým nástrojem pro reprezentaci strukturovaných dat – ať už jde o soubory, výrazy nebo organizační grafy.
- Rekurse nám umožňuje přirozeně řešit problémy, které se skládají ze stejných menších částí.
- Backtracking umí efektivně procházet kombinace a omezit výpočet jen na ty smysluplné.
- Mistrovská věta a ostatní metody (substituce, rekurzivní strom) nám pomáhají pochopit složitost rekurzivních algoritmů bez zdlouhavých výpočtů.

3. Fronta, zásobník

- Průchod stromem/grafem do šířky a do hloubky, využití fronty a zásobníku.

Fronta

Fronta je abstraktní datová struktura, která funguje na principu **FIFO** (First In, First Out).

- Prvky se vkládají na konec a odebírají se z čela.
- Lze ji implementovat např. pomocí cyklického pole.
- Typicky se používá při procházení stromů nebo grafů do **šířky** (BFS).
- Implementace obvykle obsahuje dva ukazatele – `head` a `tail`.

Zásobník

Zásobník je datová struktura s pořadím **LIFO** (Last In, First Out).

- Prvky se vkládají i odebírají z jednoho konce – z vrcholu zásobníku.
- Snadno se implementuje pomocí pole.
- Používá se např. při **rekurzi** nebo při průchodu grafu/stromu do **hloubky** (DFS).

Průchod stromem do šířky (BFS)

BFS (Breadth-First Search) prochází graf po vrstvách.

- Vytvoříme prázdnou frontu a vložíme do ní kořen.

- Dokud není fronta prázdná:
 - odebereme prvek z čela,
 - zpracujeme ho,
 - a vložíme do fronty všechny jeho potomky.
- BFS vždy najde **nejkratší cestu**, pokud existuje (počítá se počet hran).
- Nevýhodou je větší paměťová náročnost – musí uchovávat celou "vrstvu" uzlů.

Průchod stromem do hloubky (DFS)

DFS (Depth-First Search) se snaží jít co nejhlouběji, než se vrátí zpět.

- Na začátku vložíme kořen na vrchol zásobníku.
- Poté opakujeme:
 - odebereme vrchol,
 - zpracujeme ho,
 - a vložíme jeho potomky (typicky v určitém pořadí).
- DFS **nemusí najít nejkratší cestu**, ale často je **úspornější na paměť** než BFS.

IDDFS – iterativní prohledávání do hloubky

IDDFS (Iterative Deepening DFS) kombinuje výhody BFS a DFS.

- Spouštíme DFS s omezením hloubky, které postupně zvyšujeme.
- Tak najdeme nejkratší řešení (jako BFS), ale spotřebujeme méně paměti (jako DFS).

Průchody grafem (BFS/DFS)

Přístupy platí nejen pro stromy, ale i pro obecné grafy.

- Graf můžeme reprezentovat maticí sousednosti nebo seznamem sousedů.
- Oba algoritmy mají složitost $O(|V| + |E|)$ – tj. závisí na počtu vrcholů a hran.

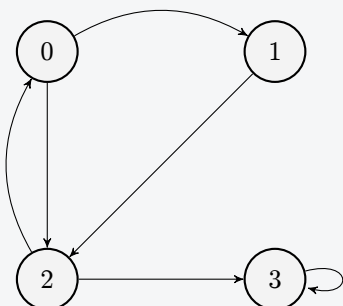
Využití v grafech:

- detekce souvislých komponent,
- detekce cyklů,
- hledání minimální kostry (spanning tree),
- hledání nejkratší cesty (BFS),
- topologické uspořádání (DFS).

Průchody binárním stromem

- DFS (do hloubky) má několik variant, které se liší v pořadí operací:
 - **PreOrder** – operaci provádíme při první návštěvě uzlu.
 - **InOrder** – operaci provádíme po návratu z levého podstromu.
 - **PostOrder** – operaci provádíme po návratu z pravého podstromu.
- BFS (do šířky) – prochází úrovně stromu pomocí fronty.

BFS a DFS průchody pro následující graf:



[1] bfs [show](#)

4. Binární vyhledávací stromy

Binární vyhledávací stromy (BST)

Binární vyhledávací strom je binární strom, ve kterém jsou uzly uspořádány podle hodnoty klíčů:

- Levý podstrom obsahuje pouze uzly s menším klíčem než má uzel.
- Pravý podstrom obsahuje pouze uzly s větším klíčem.

Každý uzel může mít maximálně dva potomky. Díky tomuto uspořádání je možné rychle vyhledávat, vkládat nebo mazat prvky – pokud je strom vyvážený, mají operace časovou složitost $O(\log n)$.

Vyhledávání v BST

- Začínáme v kořeni stromu.
- Pokud je hledaná hodnota menší než hodnota uzlu, pokračujeme do levého podstromu.
- Pokud je větší, pokračujeme do pravého.
- Pokud se rovná, našli jsme ji.
- Díky binární struktuře je cesta k hledanému uzlu jednoznačná a v ideálním případě logaritmicky dlouhá.

Vkládání

- Postupujeme podobně jako u vyhledávání.
- Nový uzel vložíme tam, kde by se měl nacházet – vždy jako list (na konec větve).
- Pokud strom připouští duplicitní klíče, vloží se do levého nebo pravého podstromu podle pravidel.

Mazání

- Pokud má uzel **žádné** potomky, prostě ho odstraníme.
- Pokud má **jeden** potomek, nahradíme jej tímto potomkem.
- Pokud má **dva** potomky:
 - Najdeme jeho **in-order předchůdce nebo nástupce** – největší uzel v levém nebo nejmenší v pravém podstromu.
 - Jeho hodnotou nahradíme hodnotu mazání, a odstraníme ho rekurzivně – bude mít nanejvýš jeden potomek.
- BST může snadno ztratit rovnováhu – pokud vkládáme vzestupně seřazená data, strom degeneruje do lineárního seznamu.

AVL stromy

AVL stromy jsou **samovyvažující** binární vyhledávací stromy. Zajišťují, že rozdíl výšky levého a pravého podstromu (tzv. **balance factor**) každého uzlu je maximálně 1.

- To zaručuje, že operace vyhledávání, vkládání a odstraňování mají časovou složitost $O(\log n)$.
- Díky tomu je výška stromu vždy $O(\log n)$ a operace jsou efektivní i v nejhorším případě.

Rotace se používají k vyvážení stromu po vkládání nebo mazání:

- **Pravá rotace (LL)** – dvě levé hrany po sobě
- **Levá rotace (RR)** – dvě pravé hrany po sobě
- **Levá-pravá rotace (LR)** – levá, pak pravá
- **Pravá-levá rotace (RL)** – pravá, pak levá

Vyhledávání

- Probíhá stejně jako v BST – AVL zajišťuje jen lepší rovnováhu.
- Časová složitost zůstává $O(\log n)$.

Vkládání

- Nový klíč vložíme jako v BST.
- Při návratu zpět kontrolujeme výšky podstromů.
- Pokud je rozdíl > 1 , provedeme vhodnou rotaci, podle tvaru nerovnováhy.
 - **LL** – pravá rotace
 - **RR** – levá rotace
 - **LR** – levá rotace na dítěti, pak pravá rotace
 - **RL** – pravá rotace na dítěti, pak levá rotace
- Po max. 1–2 rotacích je strom opět vyvážen. Vše proběhne v čase $O(\log n)$.

Mazání

- Najdeme a odstraníme klíč (případně jej nahradíme in-order předchůdcem/nástupcem).
- Při návratu nahoru testujeme balance; pokud je mimo interval $< -1, 1 >$, rotacemi jej opravíme.
- V nejhorším provedeme až $O(\log n)$ rotací

B-stromy

B-stromy jsou **vyvážené vyhledávací stromy**, které **nejsou binární** – uzly mohou mít více klíčů i potomků

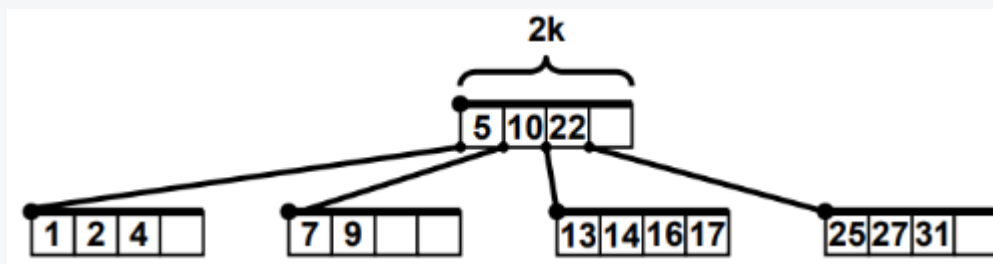
- prakticky používá v databázových systémech a souborových systémech.

B-strom je řízen parametrem **minimální stupeň t ($t \geq 2$)**:

- Každý uzel (kromě kořene) má alespoň $\lceil t/2 \rceil$ a nejvýše $2t$ klíčů.
- Z toho plyne, že každý vnitřní uzel má mezi t až $2t$ potomků.
- Kořen může mít méně než $t - 1$ klíčů.

B-stromy umožňují efektivní vyhledávání, vkládání a odstraňování uzlů. Mají tyto vlastnosti:

- Každý uzel může mít více než dva potomky.
- Všechny listy jsou na stejné úrovni.
- Každý uzel má minimální a maximální počet klíčů (viz definice t výše).
- Když uzel překročí maximální počet klíčů ($2t$), rozdělí se na dva uzly a prostřední klíč se přesune do rodiče.
- Pokud rodič také překročí maximální počet klíčů, proces se opakuje až ke kořeni.



Vyhledávání

- V každém uzlu binárně vyhledáme správný interval a pokračujeme do příslušného potomka.
- Díky větší větvenosti je výška malá – složitost $O(\log_t n)$.

Vkládání

- Klíč vkládáme do správného listu.
- Pokud uzel překročí $2t$ klíčů:
 - Rozdělíme ho na dvě části.
 - Prostřední klíč přesuneme do rodiče.

- Pokud přeteče i rodič, opakujeme postup až ke kořeni.

→ Kořen se může rozdělit – tím se strom **zvýší o úroveň**.

Mazání

- Pokud uzel klesne pod $\text{floor}(t/2)$ klíčů, musíme situaci opravit:
 - **Borrow** – vypůjčíme si klíč od sourozence.
 - **Merge** – sloučíme uzel se sourozencem a jeden klíč stáhneme z rodiče.
- Pokud se vyprázdní kořen, strom se **zmenší o úroveň**.

→ Všechny listy B-stromu jsou vždy na stejné úrovni – struktura je přirozeně vyvážená.

Srovnání stromů

Operace	BST (nevyvážený)	AVL	B-strom
_____	_____	--	_____
Vyhledávání	$O(n)$	$O(\log n)$	$O(\log n)$
Vkládání	$O(n)$	$\Theta(\log n)$	$\Theta(\log n)$
Mazání	$O(n)$	$\Theta(\log n)$	$\Theta(\log n)$
Vyváženost	ne	ano	ano

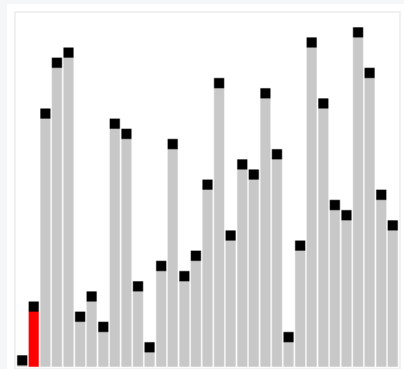
5. Algoritmy řazení

- Insert Sort, Selection Sort, Bubble Sort, QuickSort, Merge Sort, Halda, Heap Sort, Radix Sort, Counting Sort.

Insert Sort

Řadíme postupně – začínáme druhým prvkem v poli a vkládáme ho na správné místo do již seřazené části vlevo. Tímto způsobem postupně "rozšiřujeme" seřazenou oblast.

- Jednoduchý, intuitivní a stabilní algoritmus.
- Vhodný pro malá pole nebo téměř seřazené vstupy.
- Nejlepší případ (již seřazeno): $O(n)$, jinak $O(n^2)$
- In-place (nepotřebuje další paměť)



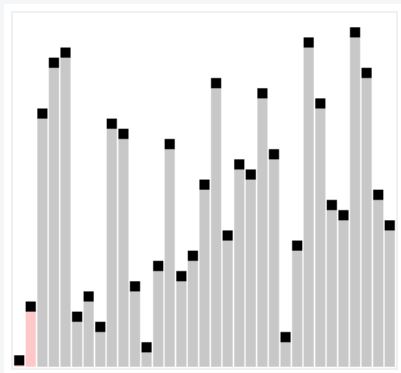
```
# Insertion Sort
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
```

```
arr[j + 1] = key
return arr
```

Selection Sort

V každé iteraci najdeme nejmenší prvek z neseřazené části a vyměníme ho s prvkem na aktuální pozici. Tímto způsobem posunujeme nejmenší prvky na začátek.

- Snadná implementace, ale **nestabilní**.
- Malý počet přepisů (výměn).
- Vždy $O(n^2)$, nezávisle na vstupu.
- In-place, ale obecně neefektivní.

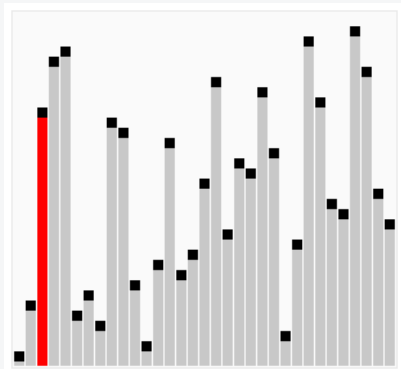


```
# Selection Sort
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Bubble Sort

Postupně procházíme pole a porovnáváme sousední prvky. Pokud jsou v nesprávném pořadí, prohodíme je. Největší prvek tak "bublá" směrem doprava.

- Stabilní a velmi jednoduchý algoritmus.
- Nevhodný pro velké množiny – vždy $O(n^2)$.
- In-place; snadná optimalizace pomocí flagu "swapped".

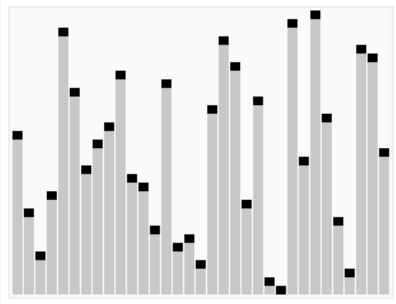


```
# Bubble Sort
def bubble_sort(arr):
    for n in range(len(arr) - 1, 0, -1):
        swapped = False
        for i in range(n):
            if arr[i] > arr[i + 1]:
                arr[i], arr[i + 1] = arr[i + 1], arr[i]
                swapped = True
        if not swapped:
            break
    return arr
```

QuickSort

Vysoce efektivní algoritmus typu "rozděl a panuj". Vybereme prvek jako pivot, rozdělíme pole na dvě části (menší a větší než pivot) a rekurzivně řadíme každou část.

- Průměrně $O(n \log n)$, ale nejhorší případ $O(n^2)$ (např. již seřazené pole a špatně zvolený pivot).
- Nestabilní, ale rychlý v praxi.
- In-place, obvykle používá rekurzi.

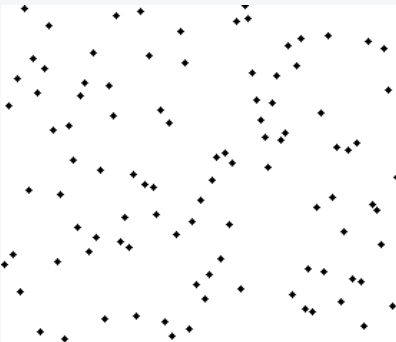


```
# QuickSort
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    less = [x for x in arr[1:] if x < pivot]
    greater_eq = [x for x in arr[1:] if x >= pivot]
    return quicksort(less) + [pivot] + quicksort(greater_eq)
```

Merge Sort

Rozdělíme pole na poloviny, každou rekurzivně seřadíme a pak sloučíme do jednoho seřazeného celku. Při slévání porovnáváme vždy první prvek obou podpolí.

- Stabilní algoritmus s garantovanou složitostí $O(n \log n)$.
- Vyžaduje pomocnou paměť pro slévání (není in-place).
- Vhodný i pro externí řazení (např. řazení souborů na disku).



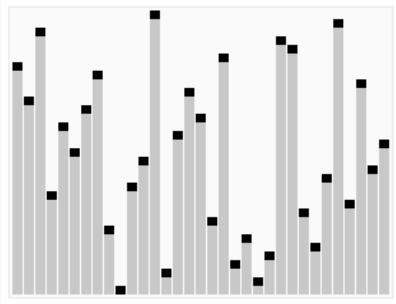
```
# Merge Sort
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Heap Sort

Vytvoříme haldu z pole (max-heap nebo min-heap), a opakovaně vyjímáme kořen (největší/nejmenší) a přesouváme ho na konec pole. Po každém vyjmutí haldu opravíme.

- Nestabilní, ale garantuje $O(n \log n)$ složitost.
- In-place – nepotřebuje pomocnou paměť.
- Využívá binární haldu; není tak rychlý jako QuickSort, ale má konzistentní výkon.



```
# Heap Sort
def heapify(arr, n, i):
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2
    if l < n and arr[l] > arr[largest]:
        largest = l
    if r < n and arr[r] > arr[largest]:
        largest = r
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
    return arr
```

Radix Sort

Řadí čísla nebo řetězce po cifrách (např. nejprve podle jednotek, pak desítek...). Používá stabilní řadicí algoritmus jako Counting Sort pro každou cifru.

- Stabilní, lineární: $O(n \cdot k)$, kde k je počet číslic/znaků.
- Vhodný pro řetězce nebo celá čísla s pevnou délkou.
- Používá se např. u PSČ, osobních čísel apod.

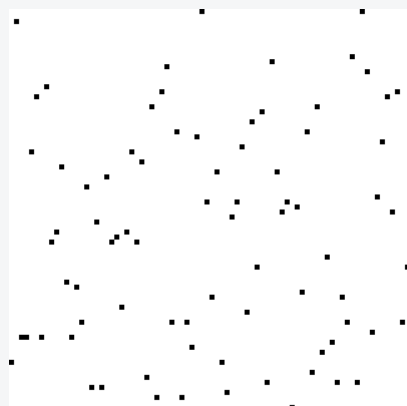
```
# Radix Sort
def counting_sort_by_digit(arr, exp):
    n = len(arr)
    output = [0] * n
    count = [0] * 10
    for i in range(n):
        index = (arr[i] // exp) % 10
        count[index] += 1
    for i in range(1, 10):
        count[i] += count[i - 1]
    for i in reversed(range(n)):
        index = (arr[i] // exp) % 10
        output[count[index] - 1] = arr[i]
        count[index] -= 1
    return output

def radix_sort(arr):
    if not arr:
        return []
    max_val = max(arr)
    exp = 1
    while max_val // exp > 0:
        arr = counting_sort_by_digit(arr, exp)
        exp *= 10
    return arr
```

Counting Sort

Nevyužívá porovnávání, ale frekvenční pole. Nejprve spočítáme výskyty všech hodnot, pak je seřadíme podle počtu. Funguje pouze pro celočíselné hodnoty v omezeném rozsahu.

- Stablní, velmi rychlý: $O(n + k)$ (k ... rozsah hodnot)
- Není in-place, vyžaduje pomocná pole.
- Vhodný pro velké vstupy s malým rozptylem hodnot.



```
# Counting Sort
def counting_sort(arr):
    if not arr:
        return []
    max_val = max(arr)
    count = [0] * (max_val + 1)
    for num in arr:
        count[num] += 1
    output = []
    for i in range(len(count)):
        output.extend([i] * count[i])
    return output
```

Shrnutí složitostí a vlastností

Algoritmus	Průměrná složitost	Stabilní	In-place	Vhodné použití
_____	_____	_____	_____	_____
Insert Sort	$O(n^2)$	Ano	Ano	Malé nebo téměř seřazené pole
Selection Sort	$O(n^2)$	Ne	Ano	Výuka, málo pohybů
Bubble Sort	$O(n^2)$	Ano	Ano	Jednoduchá implementace
QuickSort	$O(n \log n)$	Ne	Ano	Prakticky nejrychlejší
Merge Sort	$O(n \log n)$	Ano	Ne	Stabilita, externí řazení
Heap Sort	$O(n \log n)$	Ne	Ano	Stabilní výkon, bez rekurze
Counting Sort	$O(n + k)$	Ano	Ne	Hodně opakujících se čísel
Radix Sort	$O(n \cdot k)$	Ano	Ne	Řetězce/čísla s omezenou délkou

→ **Stabilita** znamená, že prvky se stejnou hodnotou zůstávají ve stejném pořadí jako na vstupu.

6. Dynamické programování

- struktura optimálního řešení, odstranění rekurse. Nejdelší společná podposloupnost, optimální násobení matic, problém batohu.

Je to všeobecná strategie pro řešení optimalizačních úloh. Významné vlastnosti:

- Hledané optimální řešení lze sestavit z vhodně volených optimálních řešení téže úlohy nad redukovanými daty.
- V rekurzivně formulovaném postupu řešení se opakovaně objevují stejné menší podproblémy. DP umožňuje obejít opakovaný výpočet většinou jednoduchou tabelací výsledků menších podproblémů, tedy volně řečeno, přepočítáním menších výsledků.

Dynamické programování pro Nejdelší Společnou Posloupnost (LCS)

- Mějme dvě posloupnosti písmen a chceme najít nejdelší společnou podposloupnost.
- Například u posloupností {BDCABA} a {ABCBADAB} je řešení {BCBA}.
- Časová složitost: $O(m \cdot n)$, kde m a n jsou délky řetězců.
- **Myšlenka algoritmu:**
 - Cíl: Najít nejdelší posloupnost znaků, která se vyskytuje ve stejném pořadí v obou vstupních řetězcích (ne nutně souvisle).
 - Dynamické programování (DP) využívá 2D tabulku pro ukládání dílčích výsledků.
 - Buňka `dp[i][j]` udává délku LCS pro první `i` znaků řetězce `X` a první `j` znaků řetězce `Y`.
 - Pokud `X[i-1] == Y[j-1]`, přidáme tento znak k LCS a přičteme 1 k `dp[i-1][j-1]`.
 - Pokud se znaky liší, vezmeme maximum z `dp[i-1][j]` (horní buňka) a `dp[i][j-1]` (levá buňka).

• Pseudokód v Pythonu:

```
def lcs(X, Y):
    m = len(X)
    n = len(Y)
    # Vytvoření DP tabulky s nulami (m+1 x n+1)
    dp = [[0]*(n+1) for _ in range(m+1)]

    # Naplnění tabulky
    for i in range(1, m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # Rekonstrukce LCS z tabulky
    i, j = m, n
    result = []
    while i > 0 and j > 0:
        if X[i-1] == Y[j-1]:
            result.append(X[i-1])
            i -= 1
            j -= 1
        elif dp[i-1][j] > dp[i][j-1]:
            i -= 1
        else:
            j -= 1
    return ''.join(reversed(result))
```

Příklad tabulky pro `X = "BDCABA"`, `Y = "ABCBADAB"`:

Řešení: **"BCBA"** (délka 4).

Tabulka:

		A	B	C	B	D	A	B
—	—	—	—	—	—	—	—	—
	0	0	0	0	0	0	0	0

B	0	0	1	1	1	1	1	1
D	0	0	1	1	1	2	2	2
C	0	0	1	2	2	2	2	2
A	0	1	1	2	2	2	3	3
B	0	1	2	2	3	3	3	4
A	0	1	2	2	3	3	4	4

• Vysvětlení tabulky:

- První řádek a sloupec jsou inicializovány na 0 (prázdné podřetězce).
- Buňka `dp[5][7]` (hodnota 4) ukazuje délku LCS.
- Cesta pro rekonstrukci je postup diagonální (při shodě znaků) nebo výběr větší hodnoty z okolních buněk.

• Složitost:

- Naplnění tabulky: $O(m \cdot n)$
- Rekonstrukce: $O(m + n)$
- Celkem: $O(m \cdot n)$

7. Rozptylovací tabulky (Hashing)

- hashovací funkce, řešení kolizí, otevřené a zřetězené tabulky, double hashing, srůstající tabulky, univerzální hashování.
- Rozptylovací tabulky – tabulka, určená k redukování dat na menší data, která budou jednoznačná.
- Základ datové struktury slovníku, kde operace search a insert mají konstantní složitost.
- Hashovací funkce – funkce která datům přiřadí nějakou hodnotu konstantní délky, která se výrazně liší pro podobná data a není z ní možné rekonstruovat originální data – hash.
- Hashovací funkce přiřadí k potenciálně libovolnému množství dat hodnotu, která má vždy konstantní délku.
- V kontextu algoritmyzace by měla být hlavně rychlá (na rozdíl od kryptografie, kde pomalost je část bezpečnosti).
- Cíl je generovat minimum kolizí.
- Kolize – situace, kdy dvěma různým datům přiřadíme stejný hash – řešíme pomocí zřetězeného rozptylování, nebo otevřeného rozptylování.
- Umožňují vyhledávat v tabulkách s průměrnou časovou složitostí $O(1)$.

Algoritmické úlohy (soutěžní)

- Two Sum – $O(n)$ hledání dvojice se zadaným součtem pomocí `set`.
- Dynamické programování s memoizací – ukládání $T(n)$ do hashe (Fibonacci, LCS...).
- Rolling hash (Rabin–Karp) – rychlé vyhledávání vzoru v textu.
- Longest Consecutive Sequence – $O(n)$ řešení s `unordered_set`.
- Počty výskytů / anagramy – frekvenční slovník pro srovnání řetězců.

Praktické použití

- Symbol table v překladači (identifikátor → metadata).
- Hash join v relačních databázích při spojování tabulek.
- In-memory cache (DNS, LRU) pro konstantní přístup.
- Deduplikace a kontrola integrity souborů (git objekty, zálohy).
- Bloom filter – pravděpodobnostní test příslušnosti s malou pamětí.

Kolize v hashovacích tabulkách

- Kolize nastávají, protože zobrazení z klíčů do adres hashovací funkce není prosté.
- Hashovací tabulky se používají k rychlému vyhledávání, ale protože hashovací funkce mohou mít kolize, tabulky musí umět tyto kolize řešit.
- Tabulka je nejčastěji reprezentovaná polem s indexy $0 \dots n - 1$, kde n je délka pole.

Existují tři běžné způsoby řešení kolizí:

- zřetězený hashing
- lineární sondování
- double hashing

Zřetězený hashing

- Každá adresa si zachovává spojový seznam všech hodnot, které se namapují na danou adresu.
- Vkládání – vypočítáme index $h(k)$. Pokud políčko už obsahuje prvek, vložíme nový na konec jeho seznamu.
- Hledání – projdeme seznam, dokud klíč nenajdeme, nebo nedojdeme na jeho konec.
- Mazání – odstraníme uzel ze seznamu (klasická operace na linked-listu).

Výhody:

- Jednoduchá implementace, žádný problém s přeplněním pole.
- Výkon se zhoršuje hladce – degraduje k $O(n)$ jen když se load-factor blíží ∞ .

Nevýhody:

- Vícenásobné alokace a ukazatele → ztráta cache-locality.
- Potenciálně vyšší režie alokátorů při častém vkládání.

Složitost:

- Average-case: $O(1)$
- Worst-case: $O(n)$

Otevřené rozptylování – Linear probing

Jak to funguje – pokud je políčko z hashu obsazeno, posouváme se lineárně dál (modulo n).

- **Vkládání** – vypočítáme index $h(k)$. Kolize $\Rightarrow$ zkusíme $(h(k) + 1) \bmod n$, pak $+1 \dots$ dokud nenajdeme prázdné místo.
- **Hledání** – stejně sondujeme, dokud
 - nenajdeme klíč (nalezen) nebo nenarazíme na prázdné políčko (klíč tam není).
- **Mazání** – prvek nelze jen tak fyzicky odstranit, musíme použít **deleted marker** nebo re-hash kolem.

Výhody

- Minimum ukazatelů $\rightarrow$ skvěle využívá cache.
- Jednoduchá implementace (jedna hash funkce, žádné seznamy).

Nevýhody

- **Primární shlukování (primary clustering)** – sousední obsazená místa tvoří dlouhé bloky a dramaticky zvyšují počet sond.
- Vyšší load-factor (> 0.7) vede k prudkému zhoršení výkonu.

Složitost

- Average-case: $O(1)$
- Worst-case: $O(n)$

Otevřené rozptylování – Double hashing

Používáme dvě hashovací funkce:

- $h1(k)$ – primární index v rozsahu $0 \dots n - 1$
- $h2(k)$ – **krok (step)** v rozsahu $1 \dots n - 1$;
 - musí být nesoudělný s n , aby prohlídka pokryla celé pole.

Algoritmus sondování $index(i) = (h1(k) + i * h2(k)) \bmod n, i = 0, 1, 2, \dots$

- **Vkládání** – sondujeme, dokud nenajdeme prázdné políčko.
- **Hledání** – sondujeme, dokud
 - nenajdeme klíč, nebo nenarazíme na prázdné políčko → klíč tam není.
- **Mazání** – opět potřebujeme *deleted marker*.

Výhody

- Menší shlukování než u lineárního i kvadratického sondování.
- Při správné volbě $h2$ projde celou tabulku bez opakování indexů.

Nevýhody

- Dvě hash funkce a nutnost hlídat nesoudělnost → o chlup složitější implementace.
- Více vypočtů hashů (na CPU to bývá zanedbatelné, ale přesto).

Složitost

- Average-case: $O(1)$
- Worst-case: $O(n)$

Srůstající (coalesced) hashing

Kombinuje výhody otevřeného rozptylování a zřetězení:

- Tabulka má vyhrazenou „sklepní“ část (**cellar**) – např. posledních 10–20 % buněk.
- Při kolizi prvek uložíme do **prvního volného** místa hledaného lineárně *odspodu* tabulky (tj. v cellar-zóně) a pomocí ukazatele jej připojíme k řetězci své původní bucket-pozice.
- Vyhledávání prochází ukazatele stejně jako u zřetězení, ale všechny uzly sedí přímo v poli, takže nedochází k fragmentaci paměti a cache-missům.

Různé varianty:

- LISCH (late insert standard coalesced hashing) – přidáváme na konec řetězce.
- EISCH (early insert...) – přidáváme na začátek řetězce.
- LICH, EICH – jako výše, ale se sklepem.
- VICH – kombinuje, kam přidat podle toho, kde řetězec končí.

Složitost

- Best-case: $O(1)$
- Average-case (při load-faktoru $\alpha < 0.8$): $O(1)$
- Worst-case: $O(n)$

Výhody

- Lepší cache-localita než externí seznamy.
- Umožní load-faktor α blízký 1, protože cellar oddálí „srůstání“ řetězců.

Nevýhody

- Složitější správa ukazatelů a mazání.
- Není-li cellar správně dimenzován, výkon degraduje na $O(n)$ podobně jako otevřené sondování.

Univerzální hashování

Myšlenka: místo jediné pevné hash-funkce zvolíme při inicializaci **náhodně** h z rodiny $\mathcal{H}$, která splňuje podmínku univerzality:

$$\forall x \neq y \quad \Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m},$$

kde m je počet bucketů.

Důsledky

- **Očekávaná** délka řetězce (nebo počet sond) je $\leq \alpha$, takže vkládání, vyhledávání i mazání běží v $O(1)$ očekávaném čase, bez ohledu na rozdělení klíčů.
- Adversář, který nezná zvolenou h , neumí zkonstruovat mnoho kolizí ($\rightarrow$ odolnost proti útokům na hash-tabulky např. v webových serverech).

Složitost

- Best / Average (v oč.) $O(1)$, protože $\mathbb{E}[\text{kolize}] \leq \alpha$.
- Worst-case: stále $O(n)$ (kdybychom si vybrali „špatnou“ funkci), ale pravděpodobnost, že se to stane, je $\leq 1/m$ a můžeme h jednoduše přelosovat.

Trace: • [b4b33alg](#)